

Интеграция LLM в программные скрипты через API

Ключевые определения (по источникам курса)

- API (Application Programming Interface) — интерфейс, через который программа автоматически обращается к внешней модели/сервису.
- Пакетный контент-анализ — поабзацная/поглавная обработка текста скриптом с сохранением ответов в структурированном виде (JSON/таблица).
- Валидация JSON и выборочный аудит — минимальные инженерно-филологические меры против масштабирования ошибок модели на весь роман.

Содержание лекции

API позволяет программе обращаться к языковой модели автоматически: скрипт отправляет фрагмент текста и получает ответ в заданном формате. Так анализ масштабируется. Вместо разового чата появляется конвейер: разбить текст на главы или абзацы → сформировать промпт → вызвать модель → сохранить результат → агрегировать таблицу → визуализировать. Для литературоведения это открывает полуавтоматический контент-анализ больших произведений.

Жёсткий формат ответа критичен. Если модель отвечает свободной прозой, результаты трудно сравнивать и стыковать с панелью. Практичный набор полей: идентификатор фрагмента; краткий тезис; список цитат-доказательств; оценка уверенности; флаги неуверенности/иронии/недостатка контекста. Такое структурирование встраивает критичность прямо в данные и облегчает выборочный аудит.

Риски пакетной обработки выше, чем у ручного диалога. Ошибка промпта размножается на весь роман. Стоимость и лимиты запросов ограничивают эксперимент. Загрузка полного текста во внешний сервис ставит этические и правовые вопросы. Накопление мелких неточностей может создать иллюзию «системного открытия». Поэтому нужны ретрай, валидация JSON, контроль пустых ответов и обязательная ручная проверка выборки.

Методологически важно логировать версию модели, параметры и текст промпта. Без этого нельзя воспроизвести результат через месяц. В учебном прототипе допустимы упрощения; в научном пайплайне необходимы прозрачность, сопоставление с независимым методом (например, с частотным или NER-контуром) и осторожность в формулировке выводов.

Связка с предыдущими темами курса очевидна: API-контур не отменяет корпусной гигиены и не заменяет стилиометрию или граф. Он добавляет семантический слой автоматической разметки, который затем должен быть интерпретирован. Лучшие проекты соединяют уровни: структура (сеть/частоты) + семантика (LLM) + close reading спорных мест.

Учебный минимум контроля качества при API-разметке: не меньше 5–10% фрагментов читаются вручную; фиксируются типы ошибок (выдуманная цитата, сдвиг смысла, игнор ограничения); по итогам корректируется промпт, а не «дообъясняется» модель. Так конвейер остаётся инструментом исследования, а не генератором непроверяемого корпуса ярлыков. Кроме того, полезно заранее ограничить длину фрагмента: слишком большой кусок провоцирует поверхностный пересказ, слишком маленький — потерю сюжетного контекста.

Вывод: интеграция LLM через API — усилитель процедуры, а не источник готового знания. Филолог проектирует конвейер так, чтобы масштаб не уничтожал проверяемость.

Вопросы для закрепления

- 1. Зачем экспортировать ответы LLM в таблицы?
- 2. Какие поля JSON полезны для поглавного анализа?
- 3. Как снизить накопление ошибок при пакетной обработке?

Литература

- 1. Prompt Engineering Guide : [электронный ресурс]. — URL: <https://www.promptingguide.ai/ru> (дата обращения: 18.09.2026).
- 2. Bird S., Klein E., Loper E. Natural Language Processing with Python : [электронный ресурс]. — URL: <https://www.nltk.org/book/> (дата обращения: 18.09.2026).
- 3. Цифровые гуманитарные исследования : монография / А. Б. Антопольский, А. А. Бонч-Осмоловская, Л. И. Бородкин [и др.]. — Красноярск : Сиб. федер. ун-т, 2023. — 272 с.